

Sistema de Percheros Inteligentes

Documentación técnica del código

Instituto Tecnológico de Costa Rica
Área Académica de Ingeniería Mecatrónica
MT-7003 Microprocesadores y Microcontroladores
I Semestre 2026

1. Resumen ejecutivo

El sistema implementa un set de tres percheros circulares inteligentes controlados por una Raspberry Pi (microprocesador) y un ESP32 (microcontrolador). La Pi maneja la base de datos de prendas, la interfaz gráfica y la lógica de los modos de uso. El ESP32 controla los actuadores: un motor a pasos que rota la base para presentar uno de los tres percheros, y tres servomotores (uno por perchero) que rotan para mostrar al usuario la prenda seleccionada.

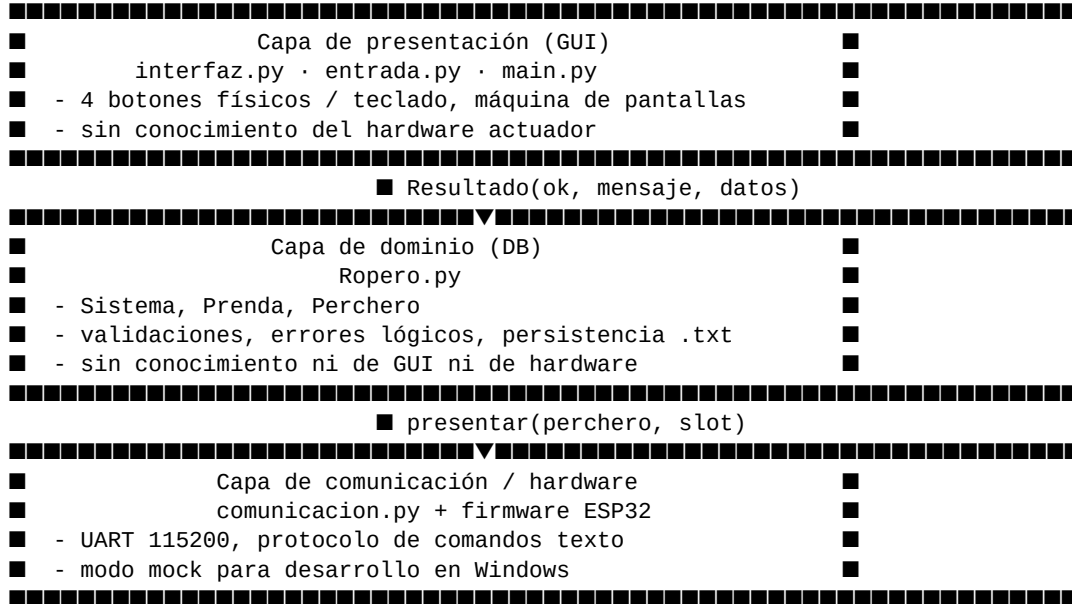
La comunicación entre Pi y ESP32 se realiza por UART (RS-232 lógico) a 115200 baud, cumpliendo el requisito del PDF del proyecto que exige que el medio de comunicación sea RS-232, I2C o SPI exclusivamente.

Estructura de archivos

Archivo	Responsabilidad	Líneas aprox.
Ropero.py	Base de datos, modelos y persistencia .txt	~430
entrada.py	Abstracción de los 4 botones (GPIO/teclado)	~140
interfaz.py	GUI Tkinter, máquina de pantallas	~810
comunicacion.py	Driver UART al ESP32, modo mock	~210
firmware_esp32.ino	Firmware Arduino del ESP32	~210
main.py	Punto de entrada: arranca todo	~55
pruebas.py	Tests funcionales de la base de datos	~190

2. Arquitectura general

El sistema se organiza en tres capas con responsabilidades disjuntas. Cada capa solo conoce a la inmediatamente inferior, lo que permite probar cada una de forma independiente y reemplazar la implementación de las capas inferiores sin tocar las superiores.



El acoplamiento es por valor: las capas se comunican mediante objetos **Resultado** (capa GUI ↔ DB) y **RespuestaESP** (capa GUI ↔ comunicación). Esto evita lanzar excepciones a través de los límites de capa y mantiene un flujo lineal y predecible.

3. Ropero.py — Base de datos

Es el corazón del sistema. Tiene cero dependencias externas (solo la biblioteca estándar de Python) y cero conocimiento de la interfaz o del hardware. Esta independencia es lo que hizo posible probar todos los modos del PDF sin Pi ni ESP32.

3.1 Constantes de dominio

Todos los conjuntos de valores válidos vienen literalmente del PDF §II. La validación se hace en el constructor de la clase Prenda; si la interfaz mete un valor fuera del dominio, la creación falla con ValueError y la GUI lo muestra como error.

```
TIPOS_PRENDA = ("T-Shirt", "Pantalón", "Short", "Camisa Manga Larga", "Enagua")
COLORES      = ("Blanco", "Negro", "Café", "Azul", "Gris")
TIPOS_TELA   = ("Denim", "Algodón", "Polyester", "Seda", "Lino")
TALLAS       = ("S", "M", "L", "XL", "XXL")
FITS         = ("Regular", "Skinny", "Slim", "Loose", "Oversized")
```

```
NUM_PERCHEROS      = 3    # PDF §II: máximo 3 percheros
SLOTS_POR_PERCHERO = 5    # PDF §II: hasta 5 prendas por perchero
```

3.2 Clase Prenda

Modela una prenda con sus 5 características obligatorias del PDF más un nombre único asignado por el usuario y un id interno auto-asignado por el sistema (no por el usuario, lo que evita colisiones).

El método **coincide(tipo, color=None, tela=None, talla=None, fit=None)** es la base de la búsqueda por características: el tipo es obligatorio (lo dice el PDF) y los demás son opcionales. Cualquier atributo None se ignora en la comparación.

3.3 Clase Perchero

Lo más importante de Perchero es que el almacenamiento es por **posición física**, no por orden de inserción:

```
self.slots: list[int|None] = [None] * 5

# slots[0] = id de la prenda en posición frontal del perchero
# slots[1] = id de la prenda a 72° de la frontal
# slots[2] = id de la prenda a 144° (...)
# slots[3] = 216°
# slots[4] = 288°
```

El índice del slot mapea directamente al ángulo que debe alcanzar el servo del perchero correspondiente. La fórmula está en el firmware del ESP32:

$$\theta(s) = 18^\circ + s \cdot 36^\circ, \quad s \in \{0, 1, 2, 3, 4\}$$

Los 18° de margen evitan que el servo choque con sus topes mecánicos. El método **colocar(prenda_id)** usa el primer slot libre y devuelve su índice; **extraer(prenda_id)** libera el slot y devuelve cuál fue. Estos índices son los que se le mandan al ESP32.

3.4 Clase Sistema

Sistema es la base de datos completa. Mantiene un diccionario de prendas indexado por id (búsqueda $O(1)$), una lista fija de 3 percheros, y un contador para auto-asignación de ids. Su API implementa un método por modo de uso del PDF:

Método	Modo del PDF	Errores que detecta
colocar_prenda_nueva	Almacenar prenda nueva	1, 2, 3
colocar_prenda_previa	Almacenar prenda previa	1, 2, 4
extraer_por_nombre	Extraer por nombre único	—
extraer_por_caracteristicas	Extraer por características	Prenda no disponible
eliminar_prenda	Eliminar prenda (interfaz)	Prenda en uso
prendas_actuales_por_perchero	Información: prendas actuales	—
prendas_conocidas_no_almacenadas	Información: prendas conocidas	—
informe_por_caracteristicas	Información: informe filtrado	—
guardar_estado / cargar_estado	Escritura y carga .txt	I/O y formato

Los 4 errores lógicos del PDF se mapean uno a uno:

#	Error	Cómo se detecta
1	No hay espacio en todos los percheros	not any(p.tiene_espacio() for p in percheros)
2	No hay espacio en el perchero específico	not perchero(target).tiene_espacio()
3	Nombre único repetido	_buscar_prenda_por_nombre(nombre) is not None
4	Prenda previa ya está guardada	_perchero_que_contiene(prenda.id) is not None

3.5 Persistencia .txt

El PDF exige guardar y cargar el estado del sistema en un archivo **.txt**. Diseñé un formato tipo INI con secciones, separador | entre campos (porque los nombres pueden tener espacios), comentarios con # y verificación de consistencia al cargar:

```
# Estado del Sistema de Percheros Inteligentes
[meta]
proximo_id=4

[prendas]
# id|nombre|tipo|color|tela|talla|fit
1|camisa-azul|Camisa Manga Larga|Azul|Algodón|M|Regular
2|jean-negro|Pantalón|Negro|Denim|L|Slim
3|short-blanco|Short|Blanco|Algodón|S|Loose

[perchero_1]
# slot_idx|prenda_id (vacío si slot vacío)
0|1
1|
```

2 | 3

3 |

4 |

Si la carga falla a la mitad (formato corrupto, ids referenciados que no existen en [prendas]) el sistema NO modifica su estado actual: construye un estado provisional y solo lo asigna si termina sin errores. Esto evita dejar la base de datos en un estado inconsistente.

4. entrada.py — Los 4 botones

Abstrae los 4 botones físicos del sistema (Sí, No, Siguiente, Anterior) detrás de una API uniforme. Cada botón tiene un único callback registrado con **entrada.on(Boton.X, funcion)**.

4.1 Detección automática de plataforma

Al iniciar, lee `/proc/device-tree/model`. Si dice "Raspberry Pi ..." asume que está en hardware real y configura los GPIO con interrupciones. En cualquier otra plataforma (Windows, macOS, Linux x86) cae automáticamente a modo simulado: las teclas de flecha y Enter/Esc generan los mismos eventos, y la GUI agrega 4 botones clickeables en la barra inferior.

4.2 Configuración GPIO en Raspberry Pi

Cada botón se conecta entre su pin GPIO y GND. La Pi activa el pull-up interno, así que el pin queda en HIGH cuando el botón está suelto y baja a LOW al presionarlo. Detectamos el flanco descendente (FALLING) con anti-rebote por software de 200 ms:

```
GPIO.setup(pin, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.add_event_detect(
    pin, GPIO.FALLING,
    callback=lambda _ch, b=boton: self.disparar(b),
    bouncetime=200,
)
```

El uso de **add_event_detect** es lo que satisface (en parte) el requisito del PDF sobre interrupciones: en lugar de hacer polling del GPIO en un loop, el kernel notifica a Python cuando el voltaje cambia. Esto es un ahorro grande de CPU y deja el thread principal libre para Tkinter.

4.3 Mapeo de pines (BCM)

Botón	GPIO BCM	Pin físico de la Pi
SI	GPIO 17	11
NO	GPIO 27	13
SIGUIENTE	GPIO 22	15
ANTERIOR	GPIO 23	16

Estos pines se eligieron para no chocar con los buses I2C (GPIO 2/3), SPI (GPIO 7-11) ni UART (GPIO 14/15) que el ESP32 necesita para comunicarse.

5. interfaz.py — GUI Tkinter

Es el archivo más grande del proyecto. Implementa la GUI completa como una **máquina de pantallas**: cada pantalla es una clase con cuatro métodos (siguiente, anterior, si, no) que el controlador invoca según qué botón se haya presionado.

5.1 Tipos de pantalla reutilizables

Clase	Para qué sirve
Selector	Lista navegable: tipo, color, perchero, prenda, modo...
Confirmacion	Pregunta sí/no antes de ejecutar una acción
Mensaje	Resultado o error, un solo botón aceptar
TecladoVirtual	Ingreso de texto con los 4 botones (cursor lineal)
DiagramaPercheros	Visualiza los 3 percheros con sus 5 slots

5.2 El stack de navegación

El controlador mantiene un stack de pantallas. Cada vez que se avanza a una nueva pantalla (push), la actual se guarda en el stack. El botón **No** hace pop: vuelve a la pantalla anterior. Esto da automáticamente un comportamiento de "atrás" en cualquier punto del flujo, sin necesidad de programar transiciones específicas.

```
def _push(self, pantalla):
    if self._actual is not None:
        self._stack.append(self._actual)
    self._mostrar(pantalla)

def _pop(self):
    if self._stack:
        self._mostrar(self._stack.pop())
    else:
        self._mostrar_menu_principal()
```

5.3 Flujo de "Colocar Prenda Nueva"

El flujo más largo del sistema. Encadena 8 pasos, cada uno presentado en su propia pantalla, usando closures (funciones anidadas) para acumular la información parcial:

```
TecladoVirtual(nombre)
→ Selector(Tipo)
→ Selector(Color)
→ Selector(Tela)
→ Selector(Talla)
→ Selector(Fit)
→ Selector(Perchero)
→ Confirmacion("¿Confirmar colocación?")
→ Sistema.colocar_prenda_nueva(...)
→ Mensaje(éxito o error)
```

En cualquier paso, presionar **No** hace pop al paso anterior, preservando todo lo que el usuario haya seleccionado hasta ese punto.

5.4 Conexión con el ESP32

Después de cada operación exitosa que afecte un slot físico (colocar / extraer / extraer por características), el método **_mostrar_resultado** revisa si el resultado tiene **datos["perchero"]** y **datos["slot"]**. Si los tiene, dispara **driver.presentar(perchero, slot)** para mover los motores. Si la mecánica falla, anexa una advertencia al mensaje pero NO invalida la operación lógica: la base de datos ya quedó consistente.

6. comunicacion.py — Driver UART

Encapsula toda la conversación con el ESP32. Tiene dos implementaciones internas que comparten la misma API pública:

Implementación	Cuándo se usa	Qué hace
_DriverSerial	Si pyserial está instalado y el puerto se abre	Manda los bytes por UART y lee la respuesta del ESP32.
_DriverMock	En Windows o si falla la apertura del puerto	Imprime cada comando en consola y simula respuestas plausibles

La selección es transparente: **ESP32Driver()** intenta abrir el UART real, y si no puede cae a mock automáticamente. Esto permite desarrollar y depurar todo en Windows sin tener el ESP32 conectado.

6.1 Protocolo de comandos

Texto plano sobre UART a 115200 baud, 8N1. Cada comando termina en \n y cada respuesta también.

Comando (Pi → ESP32)	Significado
PING	¿estás vivo?
HOME	calibrar; todos los motores a posición de origen
MOVE_BASE p	rota la base para presentar el perchero p (1..3)
ROTATE p s	el servo del perchero p gira al slot s (0..4)
PRESENT p s	MOVE_BASE p y luego ROTATE p s en una sola op.
STATUS	pide estado actual
Respuesta (ESP32 → Pi)	Significado
OK	comando ejecutado, motores en posición
BUSY	motor en movimiento, intenta luego
ERR <razón>	error (rango, sintaxis, comando inválido)
STATUS p s1 s2 s3	respuesta a STATUS

6.2 Configuración del UART

```
serial.Serial(  
    port="/dev/serial0",          # mini-UART de la Raspberry Pi  
    baudrate=115200,  
    bytesize=serial.EIGHTBITS,  
    parity=serial.PARITY_NONE,  
    stopbits=serial.STOPBITS_ONE,  
    timeout=3.0,  
)
```

El timeout de 3 segundos es generoso porque algunos comandos (MOVE_BASE, PRESENT) implican movimiento mecánico que toma cientos de milisegundos. Si pasados 3 s no hay respuesta, el driver retorna RespuestaESP(False, "Sin respuesta (timeout)") y la GUI lo muestra como advertencia mecánica.

7. firmware_esp32.ino — ESP32

Sketch Arduino que recibe comandos de la Pi por UART2 y controla el motor a pasos y los 3 servos. Es el componente que satisface el requisito de "interrupciones obligatorias" del PDF.

7.1 Interrupción de timer hardware

El generador de pulsos del motor a pasos corre en una **interrupción de timer hardware** del ESP32, configurada con:

```
timer_step = timerBegin(0, 80, true);          // timer #0, prescaler 80 → 1 µs/tick
timerAttachInterrupt(timer_step, &onStepTimer, true);
timerAlarmWrite(timer_step, STEP_HALF_PERIOD_US, true); // 625 µs
timerAlarmEnable(timer_step);
```

Cada 625 µs entra automáticamente la rutina **onStepTimer()**, que conmuta el pin STEP del driver (A4988/DRV8825). El **loop()** principal queda libre para escuchar el UART y actualizar variables. El motor se mueve sin que el firmware tenga que ocuparse del timing.

```
void IRAM_ATTR onStepTimer() {
    portENTER_CRITICAL_ISR(&timer_mux);
    if (pasos_pendientes > 0) {
        nivel_step = !nivel_step;
        digitalWrite(STEP_PIN, nivel_step ? HIGH : LOW);
        if (!nivel_step) pasos_pendientes--;
    }
    portEXIT_CRITICAL_ISR(&timer_mux);
}
```

IRAM_ATTR obliga al compilador a poner la función en memoria interna del ESP32 (no flash), porque las ISRs deben ejecutarse rápido y desde flash habría jitter. **portENTER_CRITICAL_ISR** protege la variable compartida `pasos_pendientes` frente a accesos concurrentes desde el loop.

7.2 Cálculos de movimiento

Motor a pasos (base): motor estándar $1.8^\circ/\text{paso} \times \text{microstepping } 1/8 = 0.225^\circ/\text{paso}$. Una vuelta completa son 1600 pasos. Cambiar de un perchero al siguiente son $120^\circ = 533$ pasos:

```
PASOS_POR_REV      = 200 * 8 = 1600
PASOS_POR_PERCHERO = 1600 / 3 = 533
```

A 800 Hz de pulsos (período 1.25 ms entre pulsos completos), cambiar de perchero toma $533/800 \approx 0.67$ segundos.

Servos: 5 posiciones igualmente espaciadas en el rango útil de 18° a 162° (los 18° de margen evitan los topes mecánicos):

```
 $\theta(s) = 18^\circ + s \cdot 36^\circ, \quad s \in \{0..4\}$ 
= {18°, 54°, 90°, 126°, 162°}
```

7.3 Parser de comandos UART

El loop principal lee byte a byte del UART2 y acumula en un buffer hasta encontrar `\n`. Entonces llama a `procesar_comando()`, que hace string-matching y despacha a la función correspondiente. Cada función responde por UART2 con "OK", "ERR ..." o el STATUS pedido.

8. main.py — Punto de entrada

Es el orquestador. En orden:

1. Lee argumentos CLI (`--mock`, `--puerto COM3`)
2. Crea `Sistema()` y carga `estado.txt` si existe
3. Crea `ESP32Driver` (real o mock)
4. Manda `PING + HOME` al ESP32 para validar y calibrar
5. Crea `Interfaz(sistema, driver=driver)`
6. Corre el mainloop de Tkinter
7. Al salir: guarda `estado.txt` y cierra el UART

Los argumentos de línea de comandos permiten forzar modo mock (útil para demostrar la GUI sin hardware) o especificar un puerto serial concreto (por ejemplo `COM3` en Windows si se usa un cable USB-TTL para conectar al ESP32 desde una PC).

9. Cobertura de tests

El sistema tiene tres niveles de pruebas automatizadas:

Nivel	Archivo	Qué cubre
Funcional DB	pruebas.py	Los 7 modos de uso del PDF, los 4 errores lógicos, validación de dominios, persiste
Integración GUI	Inline (mock de Tkinter)	Selector navega/confirma, push/pop del stack, flujos completos de los 4 modos del
Comunicación	Inline	PING, HOME, PRESENT, validaciones de rango cliente y servidor, que la GUI disp

10. Cumplimiento de la r brica

Criterio del PDF	Pts	Estado
Almacenaje: Prenda Nueva	15	✓ Excelente
Almacenaje: Prenda Previa	8	✓ Excelente
Almacenaje: Detecci�n de Errores	4	✓ los 4 errores
Extraer: Por Nombre �nico	8	✓ Excelente
Extraer: Por Caracter�sticas	13	Bueno (10), pendiente Excelente (req. mecanismo aparte)
Extraer: Prenda No Disponible	4	✓ Funciona
Informaci�n: Prendas Actuales (diagrama)	5	✓ Excelente
Informaci�n: Conocidas No Almacenadas	3	✓ Funciona
Informaci�n: Informe por Caracter�sticas	5	✓ Excelente
Interfaz: Existencia	3	✓ Existe
Interfaz: Eliminar Prenda	4	✓ Excelente
Interfaz: Archivo .txt de estado	10	✓ Excelente (lee y escribe)
Uso de interrupciones	4	✓ Timer HW del ESP32 + GPIO event_detect
Comunicaci�n Pi↔ESP32 RS-232/I2C/SPI	5	✓ UART a 115200
Dise�o F�sico: percheros con 5 prendas	5	✓ 5 posiciones del servo
Dise�o F�sico: cambio de perchero	7	✓ Motor a pasos rota 120�

Para alcanzar los 13 puntos del criterio "Extraer por Caracter sticas" se requiere implementar un mecanismo de entrada f sicamente distinto a la GUI. La opci n natural es un encoder rotativo o un teclado matricial conectado al ESP32, con un comando UART nuevo que el Pi consume para hacer la b squeda en la base de datos.